F.CSM302 — Үйлдлийн систем · Бие даалт 1

GitHub repository гарын авлага

Fork-Join Sorting Application

Хуваарилалтын код: **IV-2-7** · Сэдэв №7

Кодыг судлах, шүүмжлэхэд зориулсан

2026 оны 10-р сар

Гарчиг

1 Repository-гийн холбоос ба хувилбар **2**

2 Фолдерын бүтэц **2**

3 Эх кодын файл бүрийн үүрэг **3**

4 Кодыг ямар дарааллаар унших вэ **3**

5 Нэг минутад шалгах **4**

6 Commit-ийн түүх ба багийн оролцоо **4**

7 Known issues **4**

1 Repository-гийн холбоос ба хувилбар

Талбар	Утга
Холбоос	`https://github.com/<ХЭРЭГЛЭГЧ>/IV-2-7-ForkJoinSort`
Салбар (branch)	`main`
Хувилбар (tag)	`v1.0` — 9-р долоо хоногт нийтэлсэн хувилбар
Лиценз / нууц өгөгдөл	Нууц түлхүүр, нууц үг, хувийн өгөгдөл **агуулаагүй**
Гадны сан	**Байхгүй**. Зөвхөн стандарт JDK.

```
git clone https://github.com/<ХЭРЭГЛЭГЧ>/IV-2-7-ForkJoinSort.git
cd IV-2-7-ForkJoinSort
git checkout v1.0          # үнэлгээнд ашиглах тогтсон хувилбар
```

2 Фолдерын бүтэц

```
IV-2-7-ForkJoinSort/
+-- README.md              төслийн нэр, код, зорилго, Quick Start
+-- SUBMIT.md              илгээх заавар (багийн дотоод)
+-- build.sh / build.bat   компайл
+-- run.sh   / run.bat     ажиллуулах
+-- test.sh                тестийн багц
+-- src/forkjoinsort/      ЭХ КОД
+-- tests/                 58 тест
+-- examples/              оролтын жишээ файлууд
‘-- docs/                  PDF-ууд, зураг, хэмжилтийн үр дүн
```

Хэсэг	Даалгаврын шаардлага
`README.md`	Төслийн нэр, код, зорилго, Quick Start **тийм**
`src/`	Эх код **тийм**
`tests/`, `examples/`	Үндсэн ба нэмэлт тестийн өгөгдөл **тийм**
`docs/`	Manual болон шаардлагатай зураг **тийм**
`v1.0` tag	9-р долоо хоногт нийтэлсэн хувилбар **тийм**

3 Эх кодын файл бүрийн үүрэг

Файл	Мөр	Үүрэг
Алгоритмын цөм		
`MergesortAction.java`	45	`RecursiveAction` — тэнцүү хуваалт + merge
`QuicksortAction.java`	65	`RecursiveAction` — 3 талт хуваалт, merge-гүй
`SequentialSort.java`	140	insertion sort, merge, `partition3`, pivot сонголт, нэг урсгалт жишиг
Удирдлага ба хэмжилт		
`SortRunner.java`	75	тохиргоо → үр дүн. **UI-ээс бүрэн хамааралгүй**
`RunConfig.java`	55	бүх параметр + оролтын шалгалт
`RunResult.java`	30	хэмжилт, мод, зөв эсэх
`DataGenerator.java`	60	5 хэв маягийн өгөгдөл, текст оролт задлах
`Bench.java`	80	гурван хэмжилтийн туршилт (CSV)
`Main.java`	120	нэвтрэх цэг: GUI, `-headless`, `-bench`, `-render`
Модны бүртгэл		
`TraceRecorder.java`	70	**thread-safe** бүртгэл (`AtomicInteger`, `ConcurrentLinkedQueue`)
`TraceNode.java`	35	модны нэг зангилаа = нэг `compute()` дуудлага
Интерфейс		
`ui/TreePanel.java`	250	**РЕКУРСИЙН МОД** зурах самбар
`ui/MainWindow.java`	300	цонх, удирдлага, статистик, legend

4 Кодыг ямар дарааллаар унших вэ

Шүүмжлэгчид зориулсан санал болгох маршрут — **15–20 минут**:

1. `MergesortAction.java` (45 мөр) — бүх зүйлийн гол. `compute()` функц нь: threshold шалгах → хуваах → `invokeAll` → `merge`. Сурах бичгийн Project 3-ын шаардлага энд шууд харагдана.
2. `QuicksortAction.java` (65 мөр) — ижил бүтэц, гэхдээ ажил нь хуваалтын *өмнө*. Гурван талт хуваалтын ач тус (`hasL`, `hasR` шалгалт) энд.
3. `SequentialSort.partition3()` — Dijkstra-гийн гурван талт гүйлт. Яагаад pivot-ыг урьдчилан зөөхгүй байгаа тайлбарыг коммент дотор уншина уу.
4. `SortRunner.run()` — туршилтын урсгал: жишиг хэмжилт → fork-join → `Arrays.sort`-той тулгах.
5. `TraceRecorder.java` — олон урсгалаас зэрэг бичигдэх бүртгэлийг цоожгүйгээр хэрхэн аюулгүй болгосон (6–7-р бүлгийн хэрэглээ).
6. `ui/TreePanel.layoutTree()` — зангилааны дугаар нь модны бүтцийг илэрхийлэхгүй (урсгалууд зэрэг үүсгэдэг тул) учраас эцгийн дугаараар дахин барьж DFS-ээр байрлуулдаг.

Хайж байгаа зүйл	Хаанаас харах
“`RecursiveAction` мөн үү?”	`MergesortAction.java:14`, `QuicksortAction.java:16`
“`Comparable` мөн үү?”	Хоёр файлын класс зарлалт: `<T extends Comparable<? super T>`
“Threshold хаана шалгагддаг вэ?”	`compute()`-ийн эхний `if`
“Insertion sort мөн үү?”	`SequentialSort.insertionSort()`
“Оролтын шалгалт хаана вэ?”	`RunConfig.validate()`
“Зөв эсэхийг яаж шалгасан бэ?”	`SortRunner.run()` доторх `Arrays.equals(arr, expect)`

5 Нэг минутад шалгах

```
./build.sh && ./test.sh
# Хүлээгдэх: === ДҮН: 58 давсан, 0 унасан ===   (exit code 0)

./run.sh --headless -i "7,12,19,3,18,4,2,6,15,8" -t 2
# Хүлээгдэх: Зөв эсэх : ЗӨВ,  [2, 3, 4, 6, 7, 8, 12, 15, 18, 19]

./run.sh --render tree.png -n 400 -t 25 -a quick
# Дэлгэцгүй ч рекурсийн модыг зураг болгож харж болно
```

6 Commit-ийн түүх ба багийн оролцоо

Багийн гишүүн бүр өөрийн хувь нэмрийг commit-ээр баримтжуулсан. Түүхийг харах:

```
git log --oneline --graph --all
git shortlog -sn            # гишүүн бүрийн commit-ийн тоо
git log --stat v1.0         # хувилбарт орсон өөрчлөлтүүд
```

Оюутны код	Нэр	Хариуцсан ажил
B200910032	Д.Дагиймаа	UI: `TreePanel` — рекурсийн модны зураглал, зангилаа сонгох, алхамчилсан горим; **хэрэглэгчийн гарын авлага**
B221940034	Д.Оюунжаргал	`TraceRecorder`, `TraceNode`, `DataGenerator` — модны thread-safe бүртгэл, өгөгдөл үүсгэх; **README.md**
B241940014	Г.Мягмарсүрэн	`TestSuite` — 58 тест, `examples/`, хүчингүй оролтын боловсруулалт; **operational гарын авлага**
B241940127	О.Энх	Багийн ахлагч. Fork/Join цөм: `MergesortAction`, `QuicksortAction`, `SequentialSort`, `SortRunner`; нэгтгэл, тайлан, **GitHub repo / tag / release**
B242270084	Э.Бүрэнбаяр	`Bench.java` — threshold / урсгал / хэв маягийн хэмжилт, `bench.txt`, үр дүнгийн дүгнэлт; **постын хавсралт**

7 Known issues

1. Модны бүртгэл нь хурдад нөлөөлдөг тул хэмжилтийг `-no-trace` горимоор авна (`-bench` үүнийг автоматаар хийнэ).
2. $n = 10^6$, threshold = 10 үед 200 000 орчим зангилаа үүсч дүрслэл удаашрана. Дүрслэлд $n \leq 5000$ хооронд ажиллуулахыг зөвлөнө.
3. Зангилааны утгуудыг зөвхөн 48-аас бага мужид хадгална (санах ой).
4. Хурдасгалт машины цөмийн тооноос хамаарна — хөгжүүлэлтийн орчин 2 цөмтэй байсан.
5. `Comparable` шаардлагаас болж `int[]` биш `Integer[]` ашигласан (boxing-ийн зардалтай).